Curso: plugins y sistemas de aviones en C++ para X-Plane 12

Oct 12, 2026 · @Cristobal Zambrano

Introducción

Al terminar este curso sabrás escribir, compilar, instalar y depurar un plugin en C++ para X-Plane 12, y convertirlo en el "cerebro" de los sistemas de un avión propio. Partimos de cero: no hace falta haber tocado nunca el SDK de Laminar Research, solo saber lo básico de C++ (variables, funciones, punteros y compilar un proyecto).

Cómo está organizado. Cinco módulos, cada uno construido sobre el anterior:

1. **Arquitectura:** cómo X-Plane encuentra, carga y ejecuta un plugin, y dónde vive dentro de un avión.
2. **Entorno:** compiladores, SDK y una plantilla de proyecto con CMake que sirve para Windows, Linux y macOS.
3. **Hello World:** las cinco funciones obligatorias, la primera compilación y la lectura del `Log.txt`.
4. **Comunicación:** DataRefs (leer y escribir el estado del simulador), Commands (acciones) y Flight Loops (código que se ejecuta cada frame).
5. **Sistemas de aeronave:** un sistema eléctrico de ejemplo atado a un avión concreto, y las reglas de rendimiento para no provocar tirones.

Cada módulo termina con un **mini-ejercicio**. Hazlos: el SDK se aprende con las manos, no leyendo.

La analogía que usaremos todo el curso. Imagina X-Plane como un **teatro**. El simulador es el director y el escenario; tu plugin es un **actor invitado**. El actor no puede entrar en la sala de máquinas del teatro (el código interno de X-Plane): solo puede hablar con el director a través de un **intercomunicador** con botones muy concretos. Ese intercomunicador es el **SDK (XPLM)**. Todo lo que hagas, lo harás pulsando esos botones.

Convenciones del curso.

- Versión de referencia: X-Plane 12 con el **SDK 4.x** (la generación pensada para XP12). Las funciones clásicas que usamos existen desde hace años, así que el código también compila con SDK más antiguos salvo donde se indique.

- Las rutas se escriben como `X-Plane 12/Resources/plugins/...`; en Windows las barras serán `\`.
- Los nombres de DataRefs y Commands son los oficiales (`sim/...`). Para comprobar que existen en tu versión usa la herramienta **DataRefTool** (o la lista `DataRefs.txt` y `Commands.txt` que trae X-Plane en `Resources/plugins/`).
- Cuando un concepto aparece en tu proyecto real **XPWeaponFX**, lo señalo como *"En XPWeaponFX"*: así ves la teoría aplicada a código que ya conoces.

Módulo 1 · Arquitectura y anatomía de X-Plane 12

Un plugin es una librería compartida que X-Plane carga en su propio proceso y a la que llama en momentos concretos; tu código nunca "se ejecuta solo", siempre responde a una llamada del simulador.

1.1 ¿Qué es exactamente un archivo .xpl?

Un `.xpl` es una librería compartida normal con otra extensión:

Sistema	Lo que realmente es	Nombre clásico dentro del plugin
Windows	una `.dll`	`64/win.xpl`
Linux	un `.so`	`64/lin.xpl`
macOS	un `.dylib` (binario universal x86_64 + arm64)	`64/mac.xpl`

X-Plane no ejecuta tu plugin como un programa con `main()`. Lo **carga en memoria** (como haría `LoadLibrary` en Windows o `dlopen` en Linux/macOS), busca por su nombre unas funciones concretas que tú **exportas**, y las llama.

Analogía: el `.xpl` es un **cartucho de consola**. La consola (X-Plane) no sabe qué juego trae el cartucho; solo sabe que tiene unos contactos en posiciones fijas. Esos contactos son las funciones `XPluginStart`, `XPluginEnable`, etc. Si faltan, la consola rechaza el cartucho.

1.2 Cómo encuentra X-Plane los plugins

Al arrancar, X-Plane recorre carpetas conocidas y, dentro de cada subcarpeta de plugin, busca el binario de **su** sistema operativo:

```
X-Plane 12/
└── Resources/
    └── plugins/                  ← plugins GLOBALES
        ├── MiPlugin/
        │   └── 64/
        │       ├── win.xpl       ← lo carga X-Plane en Windows
        │       ├── lin.xpl       ← en Linux
        │       └── mac.xpl       ← en macOS
        └── OtroPlugin/
            └── win_x64/
                └── OtroPlugin.xpl   ← formato alternativo, también válido
```

A esta organización (una carpeta con un binario por plataforma) se le llama **"fat plugin"**. La ventaja es que distribuyes una sola carpeta y cada usuario carga el binario que le toca. Los plugins de 32 bits ya no existen: X-Plane 12 solo es de 64 bits.

Los lugares donde X-Plane busca plugins son tres:

- `Resources/plugins/` → plugins **globales**, se cargan al arrancar.
- `Aircraft/<avión>/plugins/` → plugins **de aeronave**, se cargan con el avión del usuario.
- `Custom Scenery/<paquete>/plugins/` → plugins **de escenario**, se cargan al arrancar como los globales.

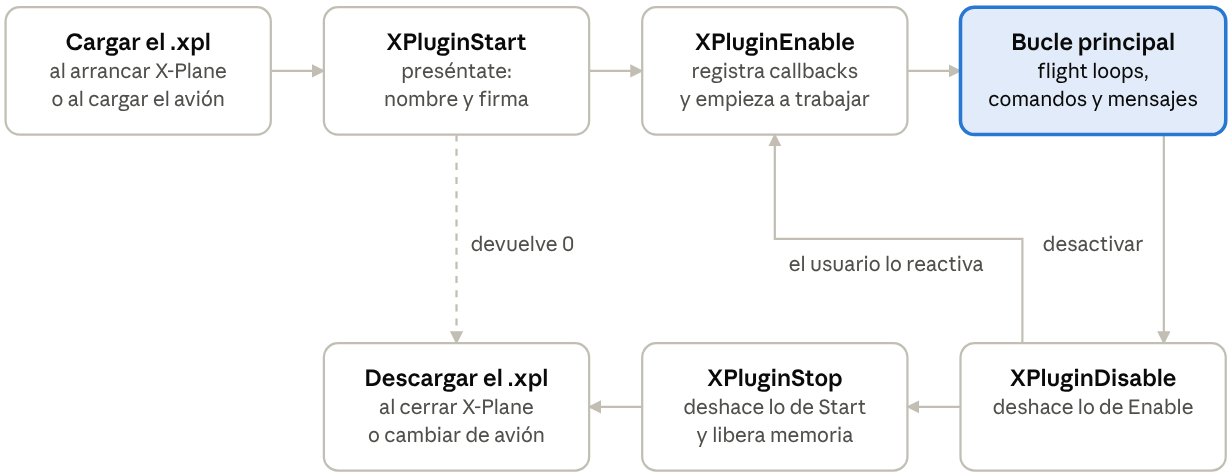
1.3 El ciclo de vida: arranque, bucle principal y cierre

Toda la vida de un plugin pasa por cuatro puertas, siempre en el mismo orden:

1. **XPluginStart** — "preséntate". X-Plane acaba de cargar la DLL. Devuelves tu nombre, firma y descripción, y preparas lo que no dependa de otros plugins. Si devuelves `0`, X-Plane te descarga.
2. **XPluginEnable** — "sal al escenario". Ya están cargados todos los plugins. Aquí registras callbacks y empiezas a trabajar. Si devuelves `0`, te quedas desactivado.
3. **Bucle principal** — X-Plane te llama cuando toca: tus *flight loops* (cada frame o cada N segundos), tus manejadores de comandos, tus callbacks de dibujo y los mensajes que te envía (`XPluginReceiveMessage`).

4. **XPluginDisable** y **XPluginStop** — "recoge y vete". Primero te desactiva (para de trabajar) y después te detiene (libera memoria). Luego descarga la DLL.

Start y Stop ocurren una vez; Enable y Disable pueden repetirse



Plugin de aeronave: el ciclo empieza al cargar el avión y termina al cambiar de avión.

ciclo de vida de un plugin · 7 pasos, 1 bucle

La línea discontinua es el atajo de un Start que devuelve 0: X-Plane descarga el plugin sin pasar nunca por Enable.

Dos ideas clave:

- **Enable/Disable pueden repetirse.** El usuario puede desactivarte y reactivarte desde *Plugins → Plugin Admin* sin descargarte. Por eso todo lo que haces en Enable debe deshacerse en Disable.
- **Start/Stop ocurren una sola vez por carga.** Lo caro (leer archivos de configuración, buscar DataRefs) va aquí.

Analogía: eres un actor invitado. *Start* es entrar al camerino y presentarte al director. *Enable* es salir al escenario. Durante la función solo hablas cuando te dan el pie (callbacks). *Disable* es salir del escenario; *Stop* es recoger tus cosas del camerino y marcharte.

1.4 Plugin global frente a plugin de aeronave

El código es el mismo; lo que cambia es **dónde** lo instalas, y con eso cambia **cuándo** vive.

	Plugin global	Plugin de aeronave
Carpeta	`Resources/plugins/<nombre>/`	`Aircraft/<avión>/plugins/<nombre>/`
Se carga	al arrancar X-Plane	cuando el usuario carga ese avión
Se descarga	al cerrar X-Plane	cuando el usuario cambia de avión (o al cerrar)
Afecta a	todo el simulador, con cualquier avión	los sistemas de ese avión
Se recarga con *Developer → Reload plugins*	sí	no: se recarga volviendo a cargar el avión
Usos típicos	herramientas, efectos, interfaces, puentes a hardware	aviónica, sistemas eléctricos e hidráulicos, FMS, armamento propio

Solo se cargan los plugins del **avión del usuario**; los aviones de IA no cargan los suyos.

En XPWeaponFX: es un plugin **global** (vive en `Resources/plugins/`). Por eso funciona con cualquier avión y repite el *discovery* con cada avión cargado. Y por eso no ve el armamento del RAZBAM A-29B: ese avión gestiona sus armas con **su propio plugin de aeronave** (A29B_HUD), al margen del sistema nativo `sim/weapons/*`.

1.5 Anatomía de la carpeta de un avión

Un avión de X-Plane es una carpeta. El simulador lee el `.acf` y, desde ahí, todo lo demás por rutas relativas:

```
Aircraft/
└── MiAvion/
    ├── MiAvion.acf            ← el "ADN" del avión, creado con Plane Maker
    ├── MiAvion_icon11.png     ← miniatura en el menú de selección
    ├── objects/               ← modelos 3D .obj: fuselaje, cabina,
animaciones
    ├── liveries/              ← pinturas alternativas
    ├── airfoils/              ← perfiles alares propios (opcional)
    ├── weapons/               ← armas propias (opcional)
    ├── fmod/                  ← bancos de sonido FMOD (opcional)
    └── plugins/
        └── MiAvionSystems/    ← TU plugin de aeronave
            └── 64/
                └── win.xpl
```

El `.acf` lo creas y editas con **Plane Maker** (viene con X-Plane). Contiene la geometría para el modelo de vuelo, los motores, la configuración de sistemas, qué objetos 3D se cargan, los paneles y los instrumentos.

La pieza clave para nosotros es esta: **los objetos 3D y la cabina se comunican con los sistemas solo a través de DataRefs y Commands**. Una aguja del voltímetro es una animación del `.obj` que lee un DataRef; un interruptor es un *manipulator* que lanza un Command o escribe un DataRef. Si tu plugin publica `miavion/elec/bus_volts`, el modelo 3D puede animar una aguja con ese valor sin saber nada de C++.

Analogía: el `.acf` y los `.obj` son la **maqueta y el cuadro de mandos** del avión; tu plugin es el **electricista** que hay detrás del panel. Los cables entre ambos son los DataRefs y los Commands, y tienen etiquetas de texto (sus nombres).

Orden práctico de carga de un avión: X-Plane carga el `.acf` y sus objetos, y después carga los plugins de `Aircraft/<avión>/plugins/` (Start → Enable). No dependas de recibir el mensaje de "avión cargado" para tu propio avión: inicializa en Enable o en el primer flight loop.

1.6 El SDK (XPLM): el intercomunicador con el simulador

El **X-Plane Plugin SDK** es un conjunto de headers en C y dos librerías:

- **XPLM**: el núcleo. Todo lo que habla con el simulador.
- **XPWidgets**: una pequeña librería de interfaz (ventanas, botones, campos de texto) construida encima de XPLM.

Es una API de **C**, no de C++, a propósito: una interfaz en C tiene un formato binario estable, así que un plugin compilado con cualquier compilador (MSVC, GCC, Clang) puede hablar con un X-Plane compilado con otro. Tú escribes C++ por dentro y llamas a funciones C por fuera.

Los headers se agrupan por tema. Los que usarás en este curso:

Header	Para qué sirve
`XPLMDefs.h`	tipos básicos y la macro `PLUGIN_API` para exportar funciones
`XPLMPlugin.h`	identidad del plugin, mensajes entre plugins, *features*
`XPLMUtilities.h`	escribir en `Log.txt`, rutas del sistema, Commands
`XPLMDataAccess.h`	leer, escribir y crear DataRefs
`XPLMProcessing.h`	flight loops (código periódico) y tiempo de simulación
`XPLMMenus.h`	añadir menús a la barra de X-Plane
`XPLMDisplay.h`	ventanas, teclado, ratón, pantallas de aviónica
`XPLMGraphics.h`	conversión de coordenadas y utilidades gráficas
`XPLMScenery.h` / `XPLMInstance.h`	sondear el terreno, cargar y colocar objetos 3D
`XPLMPlanes.h`	información de los aviones cargados (usuario e IA)

La regla de oro: **nunca toques la memoria de X-Plane directamente**. Todo pasa por XPLM. Así tu plugin sigue funcionando cuando Laminar cambia las tripas del simulador.

Mini-ejercicio 1 · Explora la anatomía de tu instalación

1. Abre X-Plane 12 y ve a *Plugins → Plugin Admin*. Apunta tres plugins cargados.
2. Cierra X-Plane y abre `X-Plane 12/Log.txt`. Busca las líneas que empiezan por `Loaded:` y localiza la ruta de esos tres plugins. Clasifica cada uno como **global**, **de aeronave** o **de escenario** según su ruta.
3. Abre la carpeta de un avión que tenga plugins (por ejemplo el A-29B o el L-159). Dibuja su árbol de carpetas como el del apartado 1.5 y marca dónde está el `.acf` y dónde el `.xpl`.

4. Pregunta de control: si instalas el mismo `win.xpl` en `Resources/plugins/` y en la carpeta de un avión, ¿cuándo se ejecuta cada copia?

Pista para el punto 4: la del avión vive solo mientras ese avión está cargado; la global vive desde el arranque hasta el cierre. Si ambas registran el mismo nombre de DataRef o Command, chocarán: por eso las firmas y los nombres deben ser únicos.

Módulo 2 · Configuración del entorno de desarrollo en C++

Necesitas tres cosas: un compilador nativo de 64 bits, el SDK de X-Plane y un proyecto que produzca una librería compartida llamada `win.xpl`, `lin.xpl` o `mac.xpl`. Con CMake, un único proyecto sirve para los tres sistemas.

Analogía: el compilador es el **taller**, el SDK es el **manual del fabricante con las medidas de los enchufes**, y CMake es el **plano de montaje** que le dice al taller qué piezas usar y con qué forma debe salir la pieza final.

2.1 Herramientas por sistema operativo

Sistema	Compilador	Cómo instalarlo	Extra
Windows	MSVC (Visual Studio 2022)	Visual Studio Community, carga de trabajo "Desarrollo para el escritorio con C++"	CMake ya viene incluido en Visual Studio
Linux	GCC o Clang	`sudo apt install build-essential cmake` (o el equivalente de tu distribución)	ninguno
macOS	Clang (Xcode)	Xcode desde la App Store y `xcode-select --install`	CMake desde cmake.org o Homebrew

Como editor vale cualquiera: Visual Studio, VS Code con la extensión CMake Tools, CLion o Xcode. Usaremos **C++17**, que todos estos compiladores soportan.

2.2 Descargar y entender el SDK

Descarga el SDK desde la página oficial de desarrolladores de X-Plane (*developer.x-plane.com → Plugin SDK → Downloads*). Elige la versión 4.x, la de X-Plane 12. Al descomprimirlo verás esto:

```
SDK/
├── CHeaders/
│   ├── XPLM/        ← headers del núcleo (XPLMDataAccess.h,
XPLMProcessing.h…)
│   ├── Widgets/     ← headers de XPWidgets (interfaz de usuario)
│   └── Wrappers/    ← envoltorios C++ antiguos; no los necesitamos
└── Libraries/
    ├── Win/         ← XPLM_64.lib y XPWidgets_64.lib (librerías de
importación)
    └── Mac/         ← XPLM.framework y XPWidgets.framework
```

Fíjate en que **no hay librerías para Linux**. No es un olvido: en Linux tu plugin se compila dejando los símbolos `XPLM...` sin resolver, y X-Plane se los proporciona en el momento de cargarlo. En Windows y macOS sí hay que enlazar contra las librerías del SDK.

Las librerías de Windows son **librerías de importación**: no contienen el código de X-Plane, solo le dicen al enlazador "estas funciones existirán en `XPLM_64.dll`, que ya está cargada dentro de X-Plane".

Macros obligatorias. Los headers del SDK necesitan saber dos cosas antes de compilar:

- **Plataforma:** exactamente una de `IBM=1` (Windows; el nombre es histórico), `APL=1` (macOS) o `LIN=1` (Linux). Si no defines ninguna, `XPLMDefs.h` da un error.
- **Versión de la API:** `XPLM200`, `XPLM210`, `XPLM300`, `XPLM301`, `XPLM400`… Cada macro "desbloquea" las funciones añadidas en esa versión del SDK. Abre `XPLMDefs.h` o la documentación de tu SDK para ver cuál es la más alta que trae (si tu SDK es 4.1 o 4.2, añade también `XPLM410`, `XPLM420`…).

Ojo con la versión: si usas una función de la API 4.x, tu plugin **solo cargará en X-Plane 12**. Para un plugin pensado solo para XP12, eso está bien.

2.3 Estructura mínima del proyecto

```
MiPrimerPlugin/
├── CMakeLists.txt        ← el plano de montaje
├── SDK/                  ← la carpeta SDK del zip, copiada tal cual
├── src/
│   └── main.cpp          ← tu código (lo escribimos en el Módulo 3)
└── dist/
    └── MiPrimerPlugin/
        └── 64/
            └── win.xpl   ← lo genera la compilación; esta carpeta se copia a
X-Plane
```

La carpeta `dist/MiPrimerPlugin` ya tiene la forma exacta que X-Plane espera, así que instalar es copiarla a `X-Plane 12/Resources/plugins/`.

2.4 El CMakeLists.txt comentado línea a línea

```
# Versión mínima de CMake. 3.20 activa por defecto las políticas modernas que
usamos abajo.
cmake_minimum_required(VERSION 3.20)

# En macOS hay que fijar las arquitecturas ANTES de project():
# x86_64 (Mac Intel) + arm64 (Apple Silicon) = binario universal.
if(APPLE)
  set(CMAKE_OSX_ARCHITECTURES "x86_64;arm64" CACHE STRING "" FORCE)
endif()

# Nombre del proyecto y lenguaje.
project(MiPrimerPlugin LANGUAGES CXX)

# Estándar de C++ obligatorio: C++17.
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Ruta al SDK (relativa a este archivo).
set(SDK_DIR "${CMAKE_CURRENT_SOURCE_DIR}/SDK")

# Una librería COMPARTIDA (.dll / .so / .dylib) con nuestro código fuente.
add_library(MiPrimerPlugin SHARED src/main.cpp)

# Dónde buscar los headers del SDK.
```

```
target_include_directories(MiPrimerPlugin PRIVATE
  "${SDK_DIR}/CHeaders/XPLM"
  "${SDK_DIR}/CHeaders/Widgets")

# Versiones de la API que queremos usar. Definir una que tu SDK no conozca es
inofensivo.
target_compile_definitions(MiPrimerPlugin PRIVATE
  XPLM200=1 XPLM210=1 XPLM300=1 XPLM301=1 XPLM400=1)

if(WIN32)
  # Plataforma Windows (IBM es el nombre histórico).
  target_compile_definitions(MiPrimerPlugin PRIVATE IBM=1 APL=0 LIN=0
    WIN32_LEAN_AND_MEAN NOMINMAX)          # evita choques con windows.h
  # Enlazar contra las librerías de importación del SDK.
  target_link_libraries(MiPrimerPlugin PRIVATE
    "${SDK_DIR}/Libraries/Win/XPLM_64.lib"
    "${SDK_DIR}/Libraries/Win/XPWidgets_64.lib")
  # Runtime de C++ estático: el usuario no necesita instalar el Visual C++
Redistributable.
  set_property(TARGET MiPrimerPlugin PROPERTY
    MSVC_RUNTIME_LIBRARY "MultiThreaded$<$<CONFIG:Debug>:Debug>")
  set(XPL_NAME win)
elseif(APPLE)
  target_compile_definitions(MiPrimerPlugin PRIVATE IBM=0 APL=1 LIN=0)
  # Ocultar todo salvo lo marcado con PLUGIN_API.
  target_compile_options(MiPrimerPlugin PRIVATE -fvisibility=hidden)
  # Enlazar contra los frameworks del SDK.
  target_link_libraries(MiPrimerPlugin PRIVATE
    "${SDK_DIR}/Libraries/Mac/XPLM.framework"
    "${SDK_DIR}/Libraries/Mac/XPWidgets.framework")
  set(XPL_NAME mac)
else()
  target_compile_definitions(MiPrimerPlugin PRIVATE IBM=0 APL=0 LIN=1)
  target_compile_options(MiPrimerPlugin PRIVATE -fPIC -fvisibility=hidden)
  # En Linux NO se enlaza nada del SDK: X-Plane aporta los símbolos al cargar
el plugin.
  set(XPL_NAME lin)
endif()

# Forma final del archivo: sin prefijo "lib", nombre win/mac/lin, extensión
.xpl,
# y directamente en dist/MiPrimerPlugin/64. El truco $<1:...> evita que
Visual Studio
```

```
# añada una subcarpeta Release/ o Debug/.
set(OUT_DIR "${CMAKE_CURRENT_SOURCE_DIR}/dist/MiPrimerPlugin/64")
set_target_properties(MiPrimerPlugin PROPERTIES
  PREFIX ""
  OUTPUT_NAME "${XPL_NAME}"
  SUFFIX ".xpl"
  RUNTIME_OUTPUT_DIRECTORY "$<1:${OUT_DIR}>"   # Windows coloca aquí la .dll
  LIBRARY_OUTPUT_DIRECTORY "$<1:${OUT_DIR}>")  # Linux y macOS colocan aquí
la .so/.dylib
```

2.5 Compilar desde la terminal

Windows (en la "Developer PowerShell for VS 2022"):

```
cmake -S . -B build -G "Visual Studio 17 2022" -A x64   # genera la solución
de 64 bits
cmake --build build --config Release                    # compila en modo
Release
```

Linux y macOS:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release   # configura en modo Release
cmake --build build -j                            # compila usando todos los
núcleos
```

Compila siempre en **Release** para probar el rendimiento real; Debug es más lento y solo sirve para depurar paso a paso.

Una nota sobre macOS: un `.xpl` que tú mismo compilas funciona sin más. Si lo distribuyes descargado de internet, macOS lo pondrá en cuarentena; para distribución pública hay que firmarlo y notarizarlo con tu cuenta de desarrollador de Apple.

En XPWeaponFX: tu proyecto ya sigue este patrón (CMake + C++17 y salida en `dist/XPWeaponFX`, que copias a mano). Es exactamente la estructura de este módulo, con más archivos fuente.

Mini-ejercicio 2 · Monta el esqueleto y compílalo vacío

1. Crea la estructura del apartado 2.3 y copia la carpeta `SDK` del zip.
2. Crea `src/main.cpp` con una sola línea: `// vacío por ahora`.
3. Configura y compila con los comandos del apartado 2.5. Debe aparecer `dist/MiPrimerPlugin/64/win.xpl` (o `lin.xpl` / `mac.xpl`).

4. Lista los símbolos exportados: en Windows `dumpbin /exports win.xpl`, en Linux `nm -D --defined-only lin.xpl`, en macOS `nm -gU mac.xpl`. Ahora la lista está vacía; guarda el comando, porque en el Módulo 3 lo repetirás y deberán aparecer las cinco funciones `XPlugin...`.
5. Pregunta de control: ¿por qué compila en Linux aunque no enlaces ninguna librería del SDK, y qué pasaría en Windows si quitaras `XPLM_64.lib`?

Pista para el punto 5: en Linux los símbolos se resuelven al cargar dentro de X-Plane. En Windows el enlazador necesita la librería de importación; sin ella, en cuanto llames a una función `XPLM...` obtendrás un error de símbolo externo sin resolver (LNK2019).

Módulo 3 · Tu primer "Hello World" como plugin en C++

Un plugin mínimo son cinco funciones exportadas con `PLUGIN_API`; con ellas X-Plane lo carga, lo activa, le envía mensajes y lo descarga. Este módulo escribe esas cinco funciones, las compila, las prueba y te enseña a leer el `Log.txt` cuando algo falla.

3.1 Las cinco funciones obligatorias

Función	Cuándo la llama X-Plane	Qué devuelves	Qué haces dentro
`XPluginStart`	una vez, justo después de cargar el `.xpl`	`1` = ok, `0` = descárgame	rellenar nombre, firma y descripción; buscar DataRefs; leer configuración
`XPluginEnable`	después de Start, y cada vez que te reactivan	`1` = ok, `0` = sigo desactivado	registrar flight loops, comandos, menús; empezar a trabajar
`XPluginReceiveMessage`	cuando ocurre algo (avión cargado, aeropuerto cargado, choque…)	nada	reaccionar a eventos del simulador o de otros plugins
`XPluginDisable`	antes de Stop, y cada vez que te desactivan	nada	deshacer todo lo de Enable

Función	Cuándo la llama X-Plane	Qué devuelves	Qué haces dentro
`XPluginStop`	una vez, antes de descargar el `.xpl`	nada	deshacer todo lo de Start y liberar memoria

Regla de simetría: lo que se crea en Start se destruye en Stop; lo que se registra en Enable se desregistra en Disable. Si la rompes, al recargar el plugin acabarás con callbacks duplicados o con X-Plane llamando a código que ya no existe (un cierre inesperado).

3.2 ¿Qué hace la macro PLUGIN_API?

En C++, el compilador "decora" los nombres de las funciones para codificar sus parámetros (*name mangling*): `XPluginStart` podría acabar llamándose algo como `?XPluginStart@@YAHPEAD00@Z`. X-Plane busca el nombre exacto `XPluginStart`, así que hay que pedir al compilador que no lo decore.

`PLUGIN_API` (definida en `XPLMDefs.h`) hace dos cosas:

- `extern "C"`: usa el nombre plano de C, sin decoración.
- **Exporta** la función: `__declspec(dllexport)` en Windows, visibilidad pública en Linux y macOS.

Analogía: es como poner tu nombre en el buzón **sin apodos ni segundos apellidos**, y además dejar el buzón **en la calle** y no dentro del jardín. El cartero (X-Plane) solo entrega si el nombre coincide letra a letra y el buzón está a la vista.

3.3 El código completo, comentado línea a línea

Guárdalo como `src/main.cpp`:

```
// ------------------------------------------------------------------
// main.cpp — "Hello World" para X-Plane 12
// ------------------------------------------------------------------
#include <cstdio>           // std::snprintf
#include <cstdint>          // intptr_t

#include "XPLMDefs.h"       // PLUGIN_API y tipos básicos del SDK
#include "XPLMPlugin.h"     // XPLMPluginID, mensajes XPLM_MSG_*,
XPLMEnableFeature
#include "XPLMUtilities.h"  // XPLMDebugString (Log.txt) y XPLMSpeakString
```

```
// Escribe una línea en Log.txt con un prefijo para encontrarla fácilmente.
static void Log(const char* msg)
{
    char buf[512];                                                  // buffer
local, sin memoria dinámica
    std::snprintf(buf, sizeof(buf), "[MiPrimerPlugin] %s\n", msg);  //
prefijo + mensaje + salto de línea
    XPLMDebugString(buf);                                           // X-
Plane lo añade a Log.txt
}

// 1) Start: X-Plane acaba de cargar el .xpl. Nos presentamos.
PLUGIN_API int XPluginStart(char* outName, char* outSig, char* outDesc)
{
    // Cada buffer de salida tiene 256 bytes. snprintf nunca se pasa de ese
tamaño.
    std::snprintf(outName, 256, "%s", "MiPrimerPlugin");              //
nombre visible en Plugin Admin
    std::snprintf(outSig,  256, "%s", "com.tunombre.miprimerplugin"); //
firma ÚNICA en todo X-Plane
    std::snprintf(outDesc, 256, "%s", "Hola mundo para X-Plane 12");  //
descripción libre

    // Pedimos rutas nativas del sistema (C:\... o /home/...) en vez del
formato antiguo de Mac.
    XPLMEnableFeature("XPLM_USE_NATIVE_PATHS", 1);

    Log("XPluginStart: cargado");
    return 1;                       // 1 = todo bien, quiero seguir cargado
}

// 2) Enable: ya están cargados todos los plugins. Empezamos a trabajar.
PLUGIN_API int XPluginEnable(void)
{
    Log("XPluginEnable: activado");
    XPLMSpeakString("Hola desde mi primer plugin");  // texto en pantalla (y
voz, si está activada)
    return 1;                       // 1 = activado correctamente
}

// 3) ReceiveMessage: X-Plane (u otro plugin) nos avisa de algo.
PLUGIN_API void XPluginReceiveMessage(XPLMPluginID inFrom, int inMsg, void*
```

```
inParam)
{
    (void)inFrom;                   // no lo usamos: evitamos el aviso del
compilador

    switch (inMsg)
    {
    case XPLM_MSG_PLANE_LOADED:
    {
        // inParam es el ÍNDICE del avión: 0 = el del usuario, 1+ = aviones
de IA.
        const int planeIndex = static_cast<int>(reinterpret_cast<intptr_t>
(inParam));
        char buf[128];
        std::snprintf(buf, sizeof(buf), "Avion cargado en el indice %d",
planeIndex);
        Log(buf);
        break;
    }
    case XPLM_MSG_AIRPORT_LOADED:   // el usuario se ha colocado en un
aeropuerto
        Log("Aeropuerto cargado");
        break;
    case XPLM_MSG_PLANE_CRASHED:    // el avión del usuario se ha estrellado
        Log("El avion se ha estrellado");
        break;
    default:                        // ignoramos el resto de mensajes
        break;
    }
}

// 4) Disable: dejamos de trabajar (deshacemos lo de Enable).
PLUGIN_API void XPluginDisable(void)
{
    Log("XPluginDisable: desactivado");
}

// 5) Stop: nos vamos (deshacemos lo de Start).
PLUGIN_API void XPluginStop(void)
{
    Log("XPluginStop: descargado");
}
```

Dos detalles que evitan problemas futuros:

- **La firma (`outSig`) debe ser única.** Usa el formato de dominio invertido (`com.empresa.plugin`). Otros plugins te encontrarán por ella con `XPLMFindPluginBySignature`, y si dos plugins comparten firma, el segundo no se carga.
- **`XPLM_USE_NATIVE_PATHS`** hace que todas las rutas que te devuelva el SDK usen el formato del sistema operativo. Actívalo siempre en Start.

Los mensajes que más usarás en `XPluginReceiveMessage`:

Mensaje	Significado	`inParam`
`XPLM_MSG_PLANE_LOADED`	se ha cargado un avión	índice del avión (0 = usuario)
`XPLM_MSG_PLANE_UNLOADED`	se ha descargado un avión	índice del avión
`XPLM_MSG_AIRPORT_LOADED`	el usuario se ha colocado en un aeropuerto	sin uso
`XPLM_MSG_SCENERY_LOADED`	ha cambiado el escenario cargado	sin uso
`XPLM_MSG_PLANE_CRASHED`	el avión del usuario se ha estrellado	sin uso
`XPLM_MSG_LIVERY_LOADED`	se ha cambiado la pintura	índice del avión

3.4 Compilar, instalar y probar

1. Compila con los comandos del Módulo 2 (apartado 2.5).
2. Comprueba las exportaciones (`dumpbin /exports`, `nm -D` o `nm -gU`): deben aparecer las cinco funciones `XPlugin...` con su nombre limpio.
3. Copia la carpeta `dist/MiPrimerPlugin` a `X-Plane 12/Resources/plugins/`. Debe quedar `Resources/plugins/MiPrimerPlugin/64/win.xpl`.
4. Arranca X-Plane y empieza un vuelo. Verás el texto "Hola desde mi primer plugin" en pantalla.
5. Abre *Plugins → Plugin Admin*: tu plugin aparece con su nombre y descripción. Desactívalo y actívalo: cada cambio deja una línea Disable/Enable en el log.

Ciclo de desarrollo rápido. El menú *Developer → Reload plugins* descarga y recarga todos los plugins globales sin reiniciar X-Plane. En Windows el `.xpl` cargado está bloqueado y no se puede sobrescribir, pero normalmente sí se puede **renombrar**: renombra `win.xpl` a `win.xpl.old`, copia el nuevo y pulsa *Reload plugins*. Si tu plugin no limpia bien en Disable/Stop, la recarga lo pondrá en evidencia.

3.5 Leer el Log.txt para depurar la carga

`Log.txt` está en la raíz de `X-Plane 12/` y se sobrescribe en cada arranque. Es tu caja negra: X-Plane apunta ahí cada plugin que carga, cada error de carga y todo lo que tú escribas con `XPLMDebugString`.

Para verlo en vivo mientras vuelas:

```
# Windows (PowerShell): muestra las últimas 30 líneas y sigue mostrando las nuevas
Get-Content "E:\Steam\steamapps\common\X-Plane 12\Log.txt" -Wait -Tail 30
```

```
# Linux / macOS
tail -f "X-Plane 12/Log.txt"
```

Una carga correcta deja una línea de este estilo, con la ruta del `.xpl` y tu firma entre paréntesis, seguida de tus propias líneas:

```
Loaded: E:\Steam\...\Resources\plugins\MiPrimerPlugin\64\win.xpl
(com.tunombre.miprimerplugin).
[MiPrimerPlugin] XPluginStart: cargado
[MiPrimerPlugin] XPluginEnable: activado
```

Cuando algo falla, el síntoma te dice dónde mirar:

Síntoma en el log	Causa probable	Solución
tu plugin no aparece en ninguna línea	estructura de carpetas mal (`.xpl` fuera de `64/`, carpeta duplicada `MiPrimerPlugin/MiPrimerPlugin/…`, nombre que no es `win.xpl`)	revisa el árbol del apartado 1.2
error de carga con código 126 (Windows)	falta una DLL de la que dependes (a menudo el runtime de Debug, `vcruntime140d.dll`)	compila en Release con el runtime estático del Módulo 2

Síntoma en el log	Causa probable	Solución
error de carga con código 193 (Windows)	el binario no es de 64 bits	genera con `-A x64`
`undefined symbol` (Linux) o `Symbol not found` (macOS)	usas una función de una API más nueva que la del X-Plane instalado, o un símbolo propio que no compilaste	revisa las macros `XPLMxxx` y los archivos fuente del CMake
`wrong architecture` / `incompatible architecture` (macOS)	binario solo Intel o solo ARM	compila universal (`x86_64;arm64`)
aparece `Loaded`, pero no tus líneas de Enable	`XPluginStart` devolvió `0`, o hay otro plugin con tu misma firma	revisa el `return` y la firma
el log termina de golpe y X-Plane se cierra	tu código ha provocado un fallo (puntero nulo, acceso fuera de un array)	la última línea `[MiPrimerPlugin]` te dice hasta dónde llegó

Técnica de las migas de pan: cuando no sepas dónde se cuelga, escribe una línea de log antes y después de cada paso sospechoso. La última miga que aparezca en el log marca el lugar del fallo. Para depurar de verdad, compila en `RelWithDebInfo` y adjunta Visual Studio (o lldb/gdb) al proceso de X-Plane.

En XPWeaponFX: los resultados de tus pruebas en el PC F: (impactos en agua y tierra, bombas detectadas) salen precisamente de líneas propias en el `Log.txt`. Esa es la técnica de este apartado aplicada a un plugin real.

Mini-ejercicio 3 · Hello World con datos reales

1. En `XPluginStart`, llama a `XPLMGetVersions(&xpVer, &sdkVer, &host)` (tres `int`, header `XPLMUtilities.h`) y escribe en el log la versión de X-Plane y la del SDK.
2. En `XPLM_MSG_PLANE_LOADED`, si el índice es `0`, llama a `XPLMGetNthAircraftModel(0, archivo, ruta)` (header `XPLMPlanes.h`; `archivo` de 256 bytes y `ruta` de 512) y escribe en el log el `.acf` cargado.
3. Recompila, instala y cambia de avión dentro de X-Plane. Comprueba que aparece una línea por cada avión cargado.

4. Sabotaje controlado: mueve `win.xpl` fuera de `64/` y arranca X-Plane. ¿Qué dice el log? Devuélvelo a su sitio y repite cambiando `return 1` por `return 0` en `XPluginStart`.

Pista: en el punto 4, el primer caso no deja rastro de tu plugin; el segundo deja la línea `Loaded` pero ninguna de tus líneas de Enable.

Módulo 4 · Comunicación con el simulador: DataRefs, Commands y Flight Loops

Todo lo que tu plugin sabe y hace pasa por tres mecanismos: los **DataRefs** describen el **estado** (cuánto, dónde, en qué posición), los **Commands** representan **acciones** (pulsar, mantener, soltar) y los **Flight Loops** deciden **cuándo** se ejecuta tu código.

Analogía: en un avión real, los DataRefs son los **indicadores y las posiciones de los interruptores**; los Commands son **el dedo que pulsa un botón**; y el Flight Loop es el **reloj de pared** que te dice "ya es hora de volver a mirar los instrumentos".

4.1 DataRefs: el estado del simulador

Un DataRef es una variable con nombre que X-Plane (u otro plugin) publica. Cada uno tiene:

- **Nombre**: una ruta de texto, por ejemplo `sim/flightmodel/position/elevation`.
- **Tipo**: entero, float, double, array de enteros, array de floats o bloque de bytes.
- **Permiso**: solo lectura o lectura y escritura.
- **Unidades**: metros, nudos, ratio 0–1… Están documentadas en `DataRefs.txt` y en DataRefTool. **Lee siempre las unidades**: la mitad de los errores de principiante son metros tomados por pies.

Trabajar con un DataRef tiene dos pasos con costes muy distintos:

1. **Buscarlo** con `XPLMFindDataRef("nombre")`. Es lento (busca por texto), así que se hace **una sola vez**, en Start o en Enable, y se guarda el *handle* que devuelve.
2. **Leerlo o escribirlo** con ese handle (`XPLMGetDataf`, `XPLMSetDataf`…). Es rápido y se puede hacer cada frame.

Analogía: buscar un DataRef es mirar un número en la guía telefónica; el handle es guardarlo en **marcación rápida**. Nadie busca en la guía cada vez que llama.

Tipo del DataRef	Leer	Escribir
`int`	`XPLMGetDatai(ref)`	`XPLMSetDatai(ref, v)`
`float`	`XPLMGetDataf(ref)`	`XPLMSetDataf(ref, v)`
`double`	`XPLMGetDatad(ref)`	`XPLMSetDatad(ref, v)`
`int[]`	`XPLMGetDatavi(ref, buf, desde, cuantos)`	`XPLMSetDatavi(ref, buf, desde, cuantos)`
`float[]`	`XPLMGetDatavf(ref, buf, desde, cuantos)`	`XPLMSetDatavf(ref, buf, desde, cuantos)`
bytes	`XPLMGetDatab(ref, buf, desde, cuantos)`	`XPLMSetDatab(ref, buf, desde, cuantos)`

Ejemplo: leer altitud, velocidad y motores, y escribir los flaps.

```
#include "XPLMDataAccess.h"

// Handles guardados en variables globales del archivo: se buscan UNA vez.
static XPLMDataRef drElevation = nullptr;  // double, metros sobre el nivel
del mar
static XPLMDataRef drIas       = nullptr;  // float, nudos indicados (piloto)
static XPLMDataRef drN1        = nullptr;  // float[], % de N1 por motor
static XPLMDataRef drFlapReq   = nullptr;  // float, posición pedida de la
palanca de flaps 0..1

static bool FindDataRefs()
{
    drElevation = XPLMFindDataRef("sim/flightmodel/position/elevation");
    drIas       =
XPLMFindDataRef("sim/cockpit2/gauges/indicators/airspeed_kts_pilot");
    drN1        =
XPLMFindDataRef("sim/cockpit2/engine/indicators/N1_percent");
    drFlapReq   = XPLMFindDataRef("sim/cockpit2/controls/flap_ratio");

    // Un nombre mal escrito devuelve nullptr. Usar un handle nulo hace
fallar el plugin.
    return drElevation && drIas && drN1 && drFlapReq;
}
```

```
static void ReadAndWriteExample()
{
    const double altM  = XPLMGetDatad(drElevation);      // lectura de un
double
    const float  iasKt = XPLMGetDataf(drIas);            // lectura de un
float

    float n1[16] = {};                                   // buffer local para
el array
    const int n = XPLMGetDatavf(drN1, n1, 0, 16);        // copia hasta 16
valores desde el índice 0
    // n = cuántos valores se copiaron de verdad (el array puede ser más
corto de lo pedido)

    // Escritura: primero comprobamos que el DataRef admite escritura.
    if (XPLMCanWriteDataRef(drFlapReq) && iasKt < 140.0f) {
        XPLMSetDataf(drFlapReq, 0.5f);                    // palanca de flaps
a la mitad
    }
    (void)altM; (void)n;                                 // en un plugin real
los usarías
}
```

Tres trampas habituales al escribir:

- **Muchos DataRefs son de solo lectura.** `XPLMSetData…` sobre ellos no hace nada. Compruébalo con `XPLMCanWriteDataRef`.
- **Algunos se pueden escribir, pero X-Plane los recalcula cada frame** y tu valor desaparece. Para tomar el control de esos sistemas existen DataRefs de anulación (`sim/operation/override/...`) que le dicen a X-Plane "esto lo calculo yo".
- **Escribe la entrada, no el resultado.** Pide los flaps con la palanca (`flap_ratio`); no fuerces el ángulo real de la superficie, que es la salida del modelo físico.

4.2 Crear tus propios DataRefs

Tu plugin también puede **publicar** DataRefs. Así la cabina 3D, otros plugins o DataRefTool pueden leer el estado de tus sistemas. X-Plane no guarda el valor: cuando alguien lo lee, llama a una función tuya (un *accessor*).

```
static float gBusVolts = 0.0f;              // el valor real vive en TU
memoria
static XPLMDataRef drBusVoltsPub = nullptr; // handle de nuestro DataRef
publicado

// X-Plane llama a esta función cada vez que alguien lee
"miavion/elec/bus_volts".
static float GetBusVolts(void* /*refcon*/) { return gBusVolts; }

static void PublishDataRefs()  // llamar en XPluginStart
{
    drBusVoltsPub = XPLMRegisterDataAccessor(
        "miavion/elec/bus_volts",  // nombre: usa tu propio prefijo, nunca
"sim/"
        xplmType_Float,            // tipo que publicamos
        0,                         // 0 = solo lectura para los demás
        nullptr, nullptr,          // int:   leer / escribir (no aplica)
        GetBusVolts, nullptr,      // float: leer = GetBusVolts / escribir =
no
        nullptr, nullptr,          // double
        nullptr, nullptr,          // int[]
        nullptr, nullptr,          // float[]
        nullptr, nullptr,          // bytes
        nullptr, nullptr);         // refcon de lectura / de escritura
}

static void UnpublishDataRefs()  // llamar en XPluginStop
{
    if (drBusVoltsPub) { XPLMUnregisterDataRef(drBusVoltsPub); drBusVoltsPub
= nullptr; }
}
```

Publica tus DataRefs en **Start**: así, cuando los demás plugins hagan su búsqueda en Enable, ya existirán. Para que aparezcan en DataRefTool, envíale su nombre al activarte:

```
// En XPluginEnable: avisar a DataRefTool de que existe nuestro DataRef.
XPLMPluginID drt = XPLMFindPluginBySignature("com.leecbaker.datareftool");
if (drt != XPLM_NO_PLUGIN_ID) {
    XPLMSendMessageToPlugin(drt, 0x01000000,
(void*)"miavion/elec/bus_volts");  // mensaje "añade este DataRef"
}
```

4.3 Commands: las acciones

Un DataRef dice **cómo está** algo; un Command dice **qué se ha hecho**. Un Command tiene tres fases, como un botón físico:

- `xplm_CommandBegin`: se ha pulsado.
- `xplm_CommandContinue`: se sigue manteniendo (llega cada frame mientras se mantiene).
- `xplm_CommandEnd`: se ha soltado.

Analogía: es el **timbre de una casa**. Un toque corto es Begin + End; dejar el dedo apretado genera Continue mientras dure. Algunos sistemas solo reaccionan al toque (subir el tren); otros necesitan que mantengas (el motor de arranque).

Puedes hacer tres cosas con los Commands:

```
#include "XPLMUtilities.h"   // XPLMFindCommand, XPLMCommandOnce,
XPLMCreateCommand…

// --- 1) EJECUTAR un comando existente ---
XPLMCommandRef cmdFlapsDown =
XPLMFindCommand("sim/flight_controls/flaps_down");
if (cmdFlapsDown) XPLMCommandOnce(cmdFlapsDown);    // Begin + End en una
sola llamada
// Para "mantener pulsado": XPLMCommandBegin(cmd) ... (más tarde) ...
XPLMCommandEnd(cmd)

// --- 2) CREAR un comando propio (aparece en Settings → Keyboard/Joystick) -
--
static XPLMCommandRef cmdBattery = nullptr;
cmdBattery = XPLMCreateCommand("miavion/elec/battery_toggle",  // nombre
único
                               "Batería: conmutar");           // descripción
visible

// --- 3) ESCUCHAR o INTERCEPTAR un comando (propio o de X-Plane) ---
static int BatteryHandler(XPLMCommandRef cmd, XPLMCommandPhase phase, void*
refcon)
{
    if (phase == xplm_CommandBegin) {   // solo reaccionamos al pulsar, no al
mantener
        // ... cambiar el estado de la batería ...
    }
```

```
    return 0;   // 0 = lo consumo: nadie más lo procesa. 1 = déjalo pasar a
los demás.
}

// En Enable:  XPLMRegisterCommandHandler(cmdBattery, BatteryHandler, 1,
nullptr);
// En Disable: XPLMUnregisterCommandHandler(cmdBattery, BatteryHandler, 1,
nullptr);
//             (mismos cuatro parámetros que al registrar, o no se
desregistra)
```

El tercer parámetro de `XPLMRegisterCommandHandler` elige si te llaman **antes** (`1`) o **después** (`0`) de que X-Plane procese el comando. Escuchar *antes* y devolver `1` te permite **observar** sin estorbar; devolver `0` te permite **bloquear** o **sustituir** el comportamiento de X-Plane (por ejemplo, impedir subir el tren con el avión en tierra).

En XPWeaponFX: así detectas el disparo del cañón: escuchas `sim/weapons/fire_guns` sin consumirlo y lo confirmas con los DataRefs `sim/weapons/firing` y la munición `sim/weapons/bul_rounds`. Es el patrón "Command para saber que se pulsó, DataRef para confirmar que pasó".

4.4 Flight Loops: tu código en cada frame

X-Plane no te deja tener un `while(true)`: si bloqueas el hilo principal, se congela todo el simulador. En su lugar, le pides que te llame periódicamente con un **Flight Loop**.

Analogía: un flight loop es un **despertador que reprogramas cada vez que suena**. Cuando te despierta, haces tu tarea rápido y le dices cuándo debe volver a sonar. El número que devuelve tu función es esa nueva alarma:

Valor devuelto	Significado
`0.5f` (positivo)	vuelve a llamarme dentro de 0,5 segundos
`-1.0f` (negativo)	vuelve a llamarme dentro de 1 frame (`-2.0f` = dentro de 2 frames)
`0.0f`	no me llames más hasta que yo vuelva a programarte

La API moderna (`XPLMCreateFlightLoop`) además te deja elegir la **fase**: antes o después del modelo de vuelo.

- **`xplm_FlightLoop_Phase_BeforeFlightModel`**: lo que escribas lo usará la física de **este** frame. Ideal para mover mandos o inyectar fuerzas.
- **`xplm_FlightLoop_Phase_AfterFlightModel`**: lees los resultados ya calculados de este frame (posición, velocidades). Ideal para instrumentos y registros.

En XPWeaponFX: tu flight loop corre en BeforeFlightModel porque X-Plane desaconseja crear instancias de objetos después del modelo de vuelo.

4.5 Ejemplo integrado: un avisador de "tren arriba y demasiado bajo"

Este plugin junta todo el módulo: lee DataRefs del simulador, publica uno propio, crea un Command para silenciar la alerta y lo evalúa todo en un flight loop a 10 Hz.

```
// ------------------------------------------------------------------
// gear_warning.cpp — avisador de tren arriba a baja altura
// ------------------------------------------------------------------
#include <cstdio>
#include "XPLMDefs.h"
#include "XPLMPlugin.h"
#include "XPLMUtilities.h"
#include "XPLMDataAccess.h"
#include "XPLMProcessing.h"

// ---- DataRefs del simulador (se buscan una vez en Start) ----
static XPLMDataRef drAgl    = nullptr;  // float, metros sobre el terreno
static XPLMDataRef drVs     = nullptr;  // float, velocidad vertical en m/s
static XPLMDataRef drGear   = nullptr;  // int, palanca del tren: 1 = abajo
static XPLMDataRef drPaused = nullptr;  // int, 1 = simulador en pausa

// ---- Lo que creamos nosotros ----
static XPLMDataRef      drWarnPub = nullptr;  // DataRef publicado:
"miplugin/gearwarn/active"
static XPLMCommandRef   cmdMute   = nullptr;  // Command:
"miplugin/gearwarn/mute"
static XPLMFlightLoopID gLoop     = nullptr;  // nuestro flight loop

// ---- Estado interno ----
static int  gActive = 0;      // 1 = alerta activa
static bool gMuted  = false;  // el piloto la ha silenciado

static void Log(const char* m) { char b[256]; std::snprintf(b, sizeof(b), "
[GearWarn] %s\n", m); XPLMDebugString(b); }
```

```
// Accessor del DataRef publicado: devuelve el estado de la alerta.
static int GetActive(void*) { return gActive; }

// Handler del Command: al pulsarlo, silencia la alerta en curso.
static int MuteHandler(XPLMCommandRef, XPLMCommandPhase phase, void*)
{
    if (phase == xplm_CommandBegin) { gMuted = true; Log("alerta
silenciada"); }
    return 0;                                          // consumido: nadie más
lo necesita
}

// El corazón del plugin: se ejecuta 10 veces por segundo.
static float WarningLoop(float /*sinceLastCall*/, float /*sinceLastLoop*/,
int /*counter*/, void*)
{
    if (XPLMGetDatai(drPaused)) return 0.1f;          // en pausa no
evaluamos nada

    const float aglFt    = XPLMGetDataf(drAgl) * 3.28084f;  // metros → pies
    const float vsFpm    = XPLMGetDataf(drVs) * 196.85f;    // m/s → pies por
minuto
    const bool  gearDown = XPLMGetDatai(drGear) == 1;

    // Histéresis: se enciende por debajo de 500 ft y se apaga por encima de
600 ft.
    // Así la alerta no "parpadea" si vuelas justo en el límite.
    if (!gActive && !gearDown && aglFt < 500.0f && vsFpm < -300.0f) {
        gActive = 1;
        if (!gMuted) XPLMSpeakString("Demasiado bajo, tren");
    } else if (gActive && (gearDown || aglFt > 600.0f)) {
        gActive = 0;
        gMuted  = false;                               // la próxima alerta
vuelve a sonar
    }
    return 0.1f;                                       // vuelve a llamarme en
0,1 s
}

PLUGIN_API int XPluginStart(char* name, char* sig, char* desc)
{
    std::snprintf(name, 256, "GearWarn");
```

```
    std::snprintf(sig,  256, "com.tunombre.gearwarn");
    std::snprintf(desc, 256, "Avisador de tren arriba a baja altura");
    XPLMEnableFeature("XPLM_USE_NATIVE_PATHS", 1);

    // 1) Buscar los DataRefs del simulador (una sola vez).
    drAgl    = XPLMFindDataRef("sim/flightmodel/position/y_agl");
    drVs     = XPLMFindDataRef("sim/flightmodel/position/vh_ind");
    drGear   = XPLMFindDataRef("sim/cockpit2/controls/gear_handle_down");
    drPaused = XPLMFindDataRef("sim/time/paused");
    if (!drAgl || !drVs || !drGear || !drPaused) { Log("falta un DataRef");
return 0; }

    // 2) Publicar nuestro DataRef (int de solo lectura).
    drWarnPub = XPLMRegisterDataAccessor("miplugin/gearwarn/active",
xplmType_Int, 0,
        GetActive, nullptr, nullptr, nullptr, nullptr, nullptr,
        nullptr, nullptr, nullptr, nullptr, nullptr, nullptr, nullptr,
nullptr);

    // 3) Crear nuestro Command.
    cmdMute = XPLMCreateCommand("miplugin/gearwarn/mute", "Silenciar aviso de
tren");
    return 1;
}

PLUGIN_API int XPluginEnable(void)
{
    // Registrar el handler del Command (antes que X-Plane: inBefore = 1).
    XPLMRegisterCommandHandler(cmdMute, MuteHandler, 1, nullptr);

    // Crear el flight loop en la fase posterior al modelo de vuelo (solo
leemos resultados).
    XPLMCreateFlightLoop_t p{};
    p.structSize   = sizeof(p);                              // obligatorio:
versión de la estructura
    p.phase        = xplm_FlightLoop_Phase_AfterFlightModel;
    p.callbackFunc = WarningLoop;
    p.refcon       = nullptr;
    gLoop = XPLMCreateFlightLoop(&p);
    XPLMScheduleFlightLoop(gLoop, 0.1f, 1);                  // primera
llamada en 0,1 s
    return 1;
}
```

```
PLUGIN_API void XPluginDisable(void)
{
    if (gLoop) { XPLMDestroyFlightLoop(gLoop); gLoop = nullptr; }      //
deshace Create
    XPLMUnregisterCommandHandler(cmdMute, MuteHandler, 1, nullptr);    //
deshace Register
    gActive = 0; gMuted = false;
}

PLUGIN_API void XPluginStop(void)
{
    if (drWarnPub) { XPLMUnregisterDataRef(drWarnPub); drWarnPub = nullptr; }
}

PLUGIN_API void XPluginReceiveMessage(XPLMPluginID, int, void*) {}
```

Fíjate en la simetría: el DataRef y el Command se crean en Start; el handler y el flight loop se registran en Enable y se destruyen en Disable; el DataRef se elimina en Stop. Los Commands creados con `XPLMCreateCommand` no se destruyen: X-Plane los conserva hasta el cierre.

Mini-ejercicio 4 · Flaps por etapas

1. Crea el Command `miplugin/flaps/next_stage`. Cada pulsación sube la palanca de flaps (`sim/cockpit2/controls/flap_ratio`) en 0,25 hasta 1,0 y después vuelve a 0.
2. Publica el DataRef `miplugin/flaps/stage` (int, 0–4) con la etapa actual.
3. Añade un flight loop a 2 Hz que impida pasar de la etapa 2 si la velocidad indicada supera 160 kt, y lo escriba en el log una sola vez por evento.
4. Asigna tu Command a una tecla en *Settings → Keyboard* y comprueba en DataRefTool que `miplugin/flaps/stage` cambia.
5. Pregunta de control: ¿por qué buscamos los DataRefs en Start y no dentro del flight loop?

Pista: `XPLMFindDataRef` busca por texto en una tabla con decenas de miles de entradas. Hacerlo 60 veces por segundo es tiempo de frame tirado; el handle no cambia mientras el plugin está cargado.

Módulo 5 · Integración de sistemas en una aeronave de X-Plane 12

Un plugin de aeronave es el mismo `.xpl` de los módulos anteriores instalado dentro de la carpeta del avión; su trabajo es simular sistemas en C++ puro y conectarlos con la cabina y con X-Plane solo a través de DataRefs y Commands, gastando una fracción de milisegundo por frame.

5.1 Vincular el plugin a tu avión

Para convertir un plugin global en uno de aeronave basta con moverlo:

```
Aircraft/MiAvion/
├── MiAvion.acf
├── objects/
└── plugins/
    └── MiAvionSystems/
        └── 64/
            ├── win.xpl
            ├── lin.xpl
            └── mac.xpl
```

A partir de ahí, X-Plane lo carga cuando el usuario elige `MiAvion` y lo descarga cuando cambia de avión. Tu plugin **es** parte del avión: puede asumir que el avión cargado es el suyo.

Casi siempre necesitarás la carpeta del avión para leer archivos propios (configuración, tablas de rendimiento, bases de datos de navegación):

```
#include <filesystem>
#include "XPLMPlanes.h"

// Devuelve la carpeta del avión del usuario (donde está el .acf).
static std::filesystem::path AircraftFolder()
{
    char file[256] = {};   // recibirá "MiAvion.acf"
    char path[512] = {};   // recibirá la ruta completa del .acf
    XPLMGetNthAircraftModel(0, file, path);   // 0 = avión del usuario
    // X-Plane entrega UTF-8; u8path lo convierte bien también en Windows con
tildes o ñ.
    return std::filesystem::u8path(path).parent_path();
}

// Uso en XPluginStart (con XPLM_USE_NATIVE_PATHS ya activado):
//   auto cfg = AircraftFolder() / "plugins" / "MiAvionSystems" /
"systems.ini";
```

Si en cambio haces un plugin **global** que debe comportarse distinto según el avión (como XPWeaponFX con el A-29B), compara el nombre del `.acf` al recibir `XPLM_MSG_PLANE_LOADED` con índice 0.

5.2 Arquitectura de un plugin de sistemas

La regla que más tiempo te ahorrará: **separa el modelo del pegamento**.

- **Modelo de sistemas**: clases C++ puras (`ElectricalSystem`, `HydraulicSystem`, `FuelSystem`…) que no incluyen ningún header del SDK. Reciben entradas y un `dt`, y calculan su estado.
- **Pegamento con X-Plane**: un único archivo que lee DataRefs, recibe Commands, llama a `Update(dt)` de cada sistema en el flight loop y publica los resultados como DataRefs.

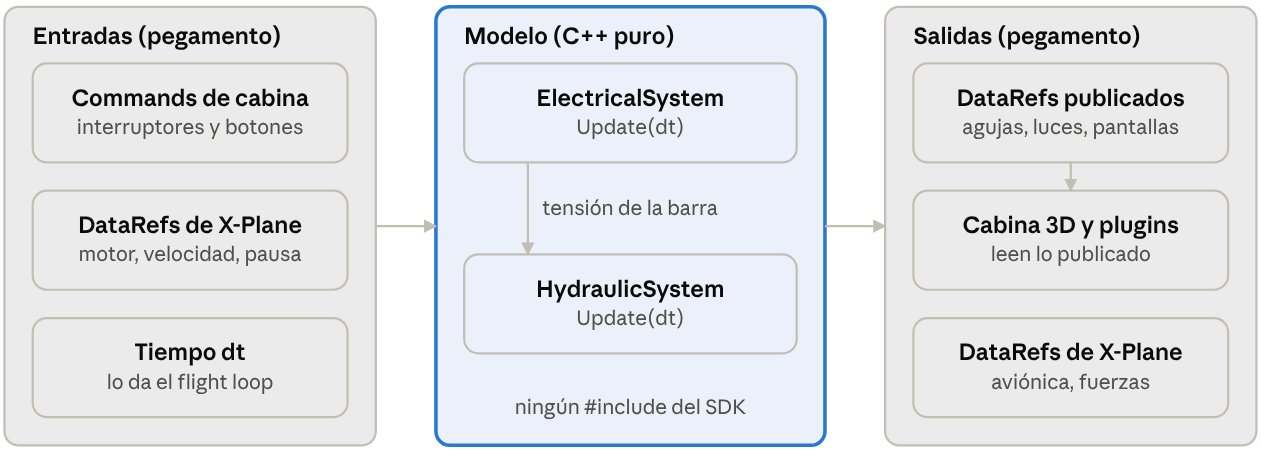
Analogía: el modelo es la **cocina** y el pegamento es el **camarero**. La cocina no habla con los clientes: recibe comandas y saca platos. El camarero toma las comandas (Commands y DataRefs de entrada), se las pasa a la cocina (`Update`) y lleva los platos a la mesa (DataRefs publicados que lee la cabina). Separados así, puedes probar la cocina con tests unitarios **sin abrir X-Plane**.

El flujo de cada frame es siempre el mismo: **entradas → modelo → salidas**.

1. **Entradas**: Commands de los interruptores de cabina, DataRefs de X-Plane (motor en marcha, velocidad, altitud) y tiempo transcurrido.

2. **Modelo**: `Update(dt)` de cada sistema, en orden de dependencia (primero eléctrico, después lo que consume electricidad).
3. **Salidas**: DataRefs publicados para agujas, luces y pantallas; y escrituras en DataRefs de X-Plane cuando tu sistema debe gobernar algo del simulador.

Solo el pegamento habla con el SDK; el modelo es C++ puro



Cada frame, el flight loop recorre las tres columnas de izquierda a derecha.

arquitectura de un plugin de sistemas · entradas, modelo, salidas

Las columnas de los lados son el pegamento con X-Plane; la del centro se puede compilar y probar sin el simulador.

5.3 Ejemplo: un sistema eléctrico completo

Primero, el **modelo**: una batería, un generador movido por el motor y una barra eléctrica con consumo. No incluye nada del SDK.

```
// ------------------------------------------------------------------
// ElectricalSystem.h — modelo eléctrico simplificado (C++ puro)
// ------------------------------------------------------------------
#pragma once
#include <algorithm>   // std::max, std::clamp

class ElectricalSystem
{
public:
    // Avanza la simulación dt segundos. engineRunning es la única entrada
externa.
    void Update(float dt, bool engineRunning)
```

```
    {
        const float genVolts  = engineRunning ? 28.0f : 0.0f;
// generador: 28 V con el motor en marcha
        const float battVolts = charge_ > 0.02f ? 22.0f + 3.0f * charge_
// batería: 22–25 V según su carga
                                                : 0.0f;
// descargada: sin tensión
        const float fromBatt  = batterySwitch_ ? battVolts : 0.0f;
// solo aporta si el interruptor está ON

        busVolts_ = std::max(genVolts, fromBatt);
// la barra toma la fuente de mayor tensión

        const float loadAmps = busVolts_ > 18.0f ? kLoadAmps : 0.0f;
// por debajo de 18 V nada funciona
        const float hours    = dt / 3600.0f;
// segundos → horas (capacidad en A·h)

        if (batterySwitch_ && genVolts > battVolts) {
            charge_ += kChargeAmps * hours / kCapacityAh;
// el generador recarga la batería
        } else if (fromBatt > 0.0f) {
            charge_ -= loadAmps * hours / kCapacityAh;
// la batería alimenta el consumo
        }
        charge_ = std::clamp(charge_, 0.0f, 1.0f);
// la carga siempre entre 0 y 1
    }

    void  ToggleBattery()       { batterySwitch_ = !batterySwitch_; }
    float BusVolts()      const { return busVolts_; }
    float BatteryCharge() const { return charge_; }
    int   BatterySwitch() const { return batterySwitch_ ? 1 : 0; }

private:
    static constexpr float kCapacityAh = 40.0f;   // capacidad de la batería
    static constexpr float kLoadAmps   = 25.0f;   // consumo total de la
barra
    static constexpr float kChargeAmps = 15.0f;   // corriente de recarga
    bool  batterySwitch_ = false;
    float charge_   = 0.9f;                       // empieza al 90 %
    float busVolts_ = 0.0f;
};
```

Después, el **pegamento**: el único archivo que conoce el SDK.

```
// ------------------------------------------------------------
// systems_plugin.cpp — conecta el modelo con X-Plane
// ------------------------------------------------------------
#include <algorithm>
#include <cstdio>
#include "XPLMDefs.h"
#include "XPLMPlugin.h"
#include "XPLMUtilities.h"
#include "XPLMDataAccess.h"
#include "XPLMProcessing.h"
#include "ElectricalSystem.h"

static ElectricalSystem gElec;                 // el modelo vive aquí, sin
memoria dinámica

// Entradas desde X-Plane
static XPLMDataRef drEngRunning = nullptr;     // int[], 1 = motor en marcha
static XPLMDataRef drPaused     = nullptr;     // int, 1 = pausa
// Salida hacia X-Plane
static XPLMDataRef drAvionics   = nullptr;     // int, aviónica de X-Plane
encendida
static int gLastAvionics = -1;                 // último valor escrito (para
escribir solo si cambia)

// Lo que publicamos
static XPLMDataRef pubBusVolts = nullptr, pubCharge = nullptr, pubBattSw =
nullptr;
static XPLMCommandRef cmdBattToggle = nullptr;
static XPLMFlightLoopID gLoop = nullptr;

// Accessors: solo devuelven valores ya calculados (deben ser baratísimos).
static float GetBusVolts(void*) { return gElec.BusVolts(); }
static float GetCharge(void*)   { return gElec.BatteryCharge(); }
static int   GetBattSw(void*)   { return gElec.BatterySwitch(); }

// Ayudantes para no repetir la larga lista de parámetros.
static XPLMDataRef PubFloat(const char* n, XPLMGetDataf_f g)
{ return XPLMRegisterDataAccessor(n, xplmType_Float, 0, nullptr, nullptr, g,
nullptr,
      nullptr, nullptr, nullptr, nullptr, nullptr, nullptr, nullptr, nullptr,
nullptr, nullptr); }
```

```
static XPLMDataRef PubInt(const char* n, XPLMGetDatai_f g)
{ return XPLMRegisterDataAccessor(n, xplmType_Int, 0, g, nullptr, nullptr,
nullptr,
      nullptr, nullptr, nullptr, nullptr, nullptr, nullptr, nullptr, nullptr,
nullptr, nullptr); }

// Command del interruptor de batería (lo lanzará el manipulador de la cabina
3D).
static int BattToggle(XPLMCommandRef, XPLMCommandPhase ph, void*)
{
    if (ph == xplm_CommandBegin) gElec.ToggleBattery();   // un cambio por
pulsación
    return 0;
}

// Entradas → modelo → salidas, una vez por frame.
static float SystemsLoop(float sinceLastCall, float, int, void*)
{
    if (XPLMGetDatai(drPaused)) return -1.0f;             // en pausa, el
tiempo del sistema no avanza

    const float dt = std::min(sinceLastCall, 0.1f);       // tras un tirón
largo no "saltamos" segundos de simulación

    int running[1] = {0};
    XPLMGetDatavi(drEngRunning, running, 0, 1);           // 1) ENTRADAS:
solo necesitamos el motor 1

    gElec.Update(dt, running[0] != 0);                    // 2) MODELO

    const int wantAvionics = gElec.BusVolts() > 18.0f ? 1 : 0;   // 3)
SALIDAS hacia X-Plane
    if (wantAvionics != gLastAvionics) {                  // escribimos solo
cuando cambia
        XPLMSetDatai(drAvionics, wantAvionics);
        gLastAvionics = wantAvionics;
    }
    return -1.0f;                                         // de nuevo en el
próximo frame
}

PLUGIN_API int XPluginStart(char* name, char* sig, char* desc)
{
```

```
    std::snprintf(name, 256, "MiAvion Systems");
    std::snprintf(sig,  256, "com.tunombre.miavion.systems");
    std::snprintf(desc, 256, "Sistemas propios de MiAvion");
    XPLMEnableFeature("XPLM_USE_NATIVE_PATHS", 1);

    drEngRunning = XPLMFindDataRef("sim/flightmodel/engine/ENGN_running");
    drPaused     = XPLMFindDataRef("sim/time/paused");
    drAvionics   =
XPLMFindDataRef("sim/cockpit2/switches/avionics_power_on");
    if (!drEngRunning || !drPaused || !drAvionics) return 0;

    pubBusVolts = PubFloat("miavion/elec/bus_volts",       GetBusVolts);
    pubCharge   = PubFloat("miavion/elec/battery_charge",  GetCharge);
    pubBattSw   = PubInt  ("miavion/elec/battery_switch",  GetBattSw);
    cmdBattToggle = XPLMCreateCommand("miavion/elec/battery_toggle",
"Batería: conmutar");
    return 1;
}

PLUGIN_API int XPluginEnable(void)
{
    XPLMRegisterCommandHandler(cmdBattToggle, BattToggle, 1, nullptr);
    XPLMCreateFlightLoop_t p{};
    p.structSize   = sizeof(p);
    p.phase        = xplm_FlightLoop_Phase_BeforeFlightModel;  // el estado
eléctrico se decide antes de la física
    p.callbackFunc = SystemsLoop;
    gLoop = XPLMCreateFlightLoop(&p);
    XPLMScheduleFlightLoop(gLoop, -1.0f, 1);
    gLastAvionics = -1;                                        // forzar la
primera escritura
    return 1;
}

PLUGIN_API void XPluginDisable(void)
{
    if (gLoop) { XPLMDestroyFlightLoop(gLoop); gLoop = nullptr; }
    XPLMUnregisterCommandHandler(cmdBattToggle, BattToggle, 1, nullptr);
}

PLUGIN_API void XPluginStop(void)
{
    for (XPLMDataRef* d : {&pubBusVolts, &pubCharge, &pubBattSw})
```

```
        if (*d) { XPLMUnregisterDataRef(*d); *d = nullptr; }
}

PLUGIN_API void XPluginReceiveMessage(XPLMPluginID, int, void*) {}
```

En un avión completo, el interruptor de aviónica del piloto también entraría en el modelo; aquí escribimos `avionics_power_on` directamente para ver el efecto sin más código.

Cómo lo usa la cabina 3D. En el `.obj` de la cabina (normalmente lo configuras desde el exportador, como XPlane2Blender) la aguja y el interruptor quedan así:

```
# Aguja del voltímetro: gira de -120° a +120° cuando bus_volts va de 0 a 30
ANIM_begin
ANIM_rotate 0 0 1  -120 120  0 30  miavion/elec/bus_volts
ANIM_end

# Interruptor de batería: un clic lanza nuestro Command
ATTR_manip_command hand miavion/elec/battery_toggle Batería
```

El artista 3D y el programador solo tienen que ponerse de acuerdo en **los nombres**. Esa es la frontera entre el `.acf` / `.obj` y tu plugin.

5.4 Otros sistemas: hidráulico, aviónica y dinámica de vuelo

- **Hidráulico, combustible, presurización:** el mismo patrón que el eléctrico. Una clase por sistema, `Update(dt)` en orden de dependencia (la bomba hidráulica eléctrica después del sistema eléctrico) y DataRefs publicados para la cabina.
- **Aviónica propia:** para pantallas que dibujas tú, el SDK 4.x de X-Plane 12 trae una API de aviónica en `XPLMDisplay.h` que te da callbacks de dibujo para tus propios dispositivos o encima de los de X-Plane. Mantén la lógica (cálculos de navegación, estados del FMS) en clases separadas y el dibujo en el pegamento.
- **Dinámica de vuelo:** ajusta primero el modelo en Plane Maker. Si necesitas más, X-Plane suma cada frame unas fuerzas y momentos adicionales que tú escribes: `sim/flightmodel/forces/faxil_plug_acf`, `fside_plug_acf`, `fnrml_plug_acf` (fuerzas, N) y `L_plug_acf`, `M_plug_acf`, `N_plug_acf` (momentos, N·m). Escríbelos en un flight loop **BeforeFlightModel**, cada frame.
- **Anulaciones (`sim/operation/override/...`):** con ellas le quitas a X-Plane un sistema entero (mandos, aceleradores…). Desde ese momento eres responsable del 100 % de ese sistema, y debes desactivar la anulación en `XPluginDisable`, o el avión quedará "muerto" si tu plugin se desactiva.

5.5 Rendimiento: cómo no provocar tirones

A 60 FPS, X-Plane tiene **16,7 ms** por frame para física, escenario, render y todos los plugins juntos. Si tu flight loop tarda 5 ms un solo frame, el usuario lo nota como un tirón. Un plugin de sistemas bien hecho debería gastar **una fracción de milisegundo**.

Analogía: el hilo principal de X-Plane es un **carril de un solo coche**. Mientras tu código ocupa el carril, nadie más avanza: ni la física ni el render. Tu trabajo es pasar rápido y apartarte.

Las reglas, de más a menos importantes:

1. **Busca DataRefs y Commands una sola vez** (Start/Enable) y guarda los handles.
2. **Nada de E/S en el bucle:** ni archivos, ni red, ni `XPLMDebugString` cada frame. Registra solo eventos o resúmenes periódicos.
3. **Nada de memoria dinámica en el bucle:** reserva vectores y buffers en Start (`reserve`), evita `new`, concatenar `std::string` o hacer crecer contenedores en cada frame.
4. **Elige la frecuencia:** no todo necesita 60 Hz. Una alerta va bien a 10 Hz y un registro a 1 Hz. Devuelve `0.1f` en vez de `-1.0f` cuando puedas.
5. **Lee arrays de una vez:** una llamada `XPLMGetDatavf` con todo el array, no un bucle elemento a elemento.
6. **Escribe solo cuando cambia** (como `gLastAvionics` en el ejemplo): cada escritura cuesta y puede disparar trabajo en otros plugins.
7. **Accessors baratísimos:** la cabina 3D puede leer tus DataRefs muchas veces por frame. Devuelve valores precalculados; nunca calcules dentro del accessor.
8. **Carga recursos de forma asíncrona:** usa `XPLMLoadObjectAsync` en vez de `XPLMLoadObject`, y coloca los objetos con `XPLMInstance` (el dibujo 3D clásico con callbacks no es el camino en X-Plane 12 con Vulkan/Metal).
9. **Trabajo pesado en otro hilo, pero el SDK solo en el principal.** XPLM **no** es seguro entre hilos: ninguna función `XPLM...` puede llamarse fuera del hilo principal.

Para la regla 9, el patrón es pasar copias de datos entre hilos:

```
#include <atomic>
#include <chrono>
#include <mutex>
#include <thread>

struct Input  { double lat = 0, lon = 0; };   // copia de lo que el hilo
```

```
necesita (nunca handles del SDK)
struct Output { float result = 0; bool ready = false; };

static std::mutex        gMx;
static Input             gIn;                // escrito por el hilo
principal
static Output            gOut;               // escrito por el hilo de
trabajo
static std::atomic<bool> gQuit{false};
static std::thread       gWorker;

static void WorkerMain()                     // NUNCA llama a funciones XPLM
{
    while (!gQuit) {
        Input in;
        { std::lock_guard<std::mutex> lk(gMx); in = gIn; }        // copia la
entrada
        const float r = static_cast<float>(in.lat * 0.5);        // ...
cálculo pesado aquí ...
        { std::lock_guard<std::mutex> lk(gMx); gOut = {r, true}; } // publica
el resultado
        std::this_thread::sleep_for(std::chrono::milliseconds(50));
    }
}

// En el flight loop (hilo principal): try_lock, para no esperar nunca al
otro hilo.
static void ExchangeWithWorker(double lat, double lon)
{
    std::unique_lock<std::mutex> lk(gMx, std::try_to_lock);
    if (!lk.owns_lock()) return;             // ocupado: lo intentamos el
próximo frame
    gIn = {lat, lon};                        // nueva entrada
    if (gOut.ready) { /* usar gOut.result */ gOut.ready = false; }
}

// Enable:  gQuit = false; gWorker = std::thread(WorkerMain);
// Disable: gQuit = true;  if (gWorker.joinable()) gWorker.join();  // el
hilo NUNCA sobrevive al plugin
```

Mide, no adivines. Cronometra tu flight loop y escribe un resumen cada 10 segundos:

```
#include <chrono>

static double gSumMs = 0, gMaxMs = 0; static int gN = 0;

static float MeasuredLoop(float dt, float a, int c, void* r)
{
    const auto t0 = std::chrono::steady_clock::now();
    const float next = SystemsLoop(dt, a, c, r);                        // el
trabajo real
    const double ms = std::chrono::duration<double, std::milli>(
                          std::chrono::steady_clock::now() - t0).count();
    gSumMs += ms; gMaxMs = std::max(gMaxMs, ms); ++gN;
    if (gN == 600) {                                                    // ~10
s a 60 FPS
        char b[128];
        std::snprintf(b, sizeof(b), "[Systems] media %.3f ms, maximo %.3f
ms\n", gSumMs / gN, gMaxMs);
        XPLMDebugString(b);                                             // un
log cada 10 s, no cada frame
        gSumMs = gMaxMs = 0; gN = 0;
    }
    return next;
}
```

Si la media pasa de unas décimas de milisegundo o el máximo tiene picos, busca el culpable con esta misma técnica dentro de cada `Update`.

Mini-ejercicio 5 · Bomba hidráulica eléctrica y prueba de estrés

1. Instala el plugin de sistemas en una **copia** de un avión (nunca en el original) y añade a la cabina, o a DataRefTool, la lectura de `miavion/elec/bus_volts`.
2. Crea `HydraulicSystem` (C++ puro): una bomba eléctrica que solo funciona con la barra por encima de 22 V y que sube la presión hasta 3000 psi con una constante de tiempo de 2 s. Publica `miavion/hyd/pressure_psi`.
3. Llama a los `Update` en el orden correcto y explica por qué ese orden importa.
4. Envuelve el flight loop con `MeasuredLoop` y anota la media y el máximo.
5. Prueba de estrés: añade temporalmente `std::this_thread::sleep_for(std::chrono::milliseconds(8))` dentro del loop. Vuela 30 segundos, observa los tirones y el informe de tiempos, y quítalo.

Pista para el punto 3: si actualizas el hidráulico antes que el eléctrico, la bomba usa la tensión del frame anterior. A 60 Hz apenas se nota, pero con dependencias en cadena el retraso se acumula y aparecen oscilaciones.

Anexo · Chuleta, errores típicos y siguientes pasos

Chuleta de la API usada en el curso

Función	Header	Para qué
`XPLMDebugString`	`XPLMUtilities.h`	escribir en `Log.txt`
`XPLMSpeakString`	`XPLMUtilities.h`	mostrar (y decir) un texto en el simulador
`XPLMGetVersions`	`XPLMUtilities.h`	versión de X-Plane y del SDK
`XPLMEnableFeature`	`XPLMPlugin.h`	activar `XPLM_USE_NATIVE_PATHS` y otras *features*
`XPLMFindPluginBySignature` / `XPLMSendMessageToPlugin`	`XPLMPlugin.h`	hablar con otros plugins
`XPLMFindDataRef`	`XPLMDataAccess.h`	obtener el handle de un DataRef (una vez)
`XPLMGetData{i,f,d}` / `XPLMSetData{i,f,d}`	`XPLMDataAccess.h`	leer y escribir valores sueltos
`XPLMGetDatav{i,f}` / `XPLMSetDatav{i,f}`	`XPLMDataAccess.h`	leer y escribir arrays
`XPLMCanWriteDataRef`	`XPLMDataAccess.h`	saber si un DataRef admite escritura
`XPLMRegisterDataAccessor` / `XPLMUnregisterDataRef`	`XPLMDataAccess.h`	publicar y retirar DataRefs propios
`XPLMFindCommand` / `XPLMCommandOnce` / `XPLMCommandBegin` / `XPLMCommandEnd`	`XPLMUtilities.h`	ejecutar Commands

Función	Header	Para qué
`XPLMCreateCommand`	`XPLMUtilities.h`	crear Commands propios
`XPLMRegisterCommandHandler` / `XPLMUnregisterCommandHandler`	`XPLMUtilities.h`	escuchar o interceptar Commands
`XPLMCreateFlightLoop` / `XPLMScheduleFlightLoop` / `XPLMDestroyFlightLoop`	`XPLMProcessing.h`	código periódico
`XPLMGetNthAircraftModel`	`XPLMPlanes.h`	`.acf` y carpeta de un avión cargado
`XPLMLoadObjectAsync` / `XPLMCreateInstance`	`XPLMScenery.h` / `XPLMInstance.h`	cargar y colocar objetos 3D

Los diez errores más habituales

1. Olvidar `PLUGIN_API` en una de las cinco funciones: X-Plane no la encuentra y no carga el plugin.
2. No definir `IBM`, `APL` o `LIN`: error de compilación en `XPLMDefs.h`.
3. Dejar el `.xpl` fuera de `64/` o con otro nombre distinto de `win.xpl`, `lin.xpl` o `mac.xpl`.
4. Usar un handle de DataRef nulo porque el nombre estaba mal escrito: comprueba siempre el `nullptr`.
5. Confundir unidades (metros con pies, m/s con nudos, radianes con grados).
6. Romper la simetría Enable/Disable: callbacks duplicados al recargar o cierres al desactivar.
7. Llamar a `XPLMFindDataRef` dentro del flight loop.
8. Llamar a cualquier función `XPLM...` desde un hilo que no es el principal.
9. Dejar activa una anulación `sim/operation/override/...` al desactivar el plugin.
10. Compilar en Debug con el runtime dinámico y distribuirlo: en otros PC falta `vcruntime140d.dll` y la carga falla.

Siguientes pasos

- **Documentación oficial:** la referencia de cada header y los ejemplos de código están en *developer.x-plane.com* (sección Plugin SDK). Compárala siempre con los headers de la versión que hayas descargado.
- **Herramientas imprescindibles:** DataRefTool (explorar y editar DataRefs y Commands en vivo) y el propio `Log.txt`.
- **Temas para profundizar:** menús y ventanas (`XPLMMenus.h`, `XPLMDisplay.h`), la API de aviónica de XP12, `XPLMInstance` para efectos y objetos, sonido con FMOD y comunicación entre plugins.
- *En XPWeaponFX:* el punto pendiente del A-29B es un caso de comunicación entre plugins. Las dos vías del curso son buscar DataRefs o Commands que publique el plugin A29B_HUD (DataRefTool con el A-29B cargado), o acordar una API compartida con mensajes `XPLMSendMessageToPlugin` o DataRefs propios.

Nota: este curso está redactado sin consultar la documentación en línea. Antes de publicar un plugin, verifica en los headers de tu SDK y en DataRefTool los nombres exactos de las funciones, DataRefs y Commands.